

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ



BÀI TIỂU LUẬN

MÔN HỌC

**ĐẢM BẢO CHẤT LƯỢNG VÀ
KIỂM THỬ PHẦN MỀM**

GVHD : TS. Nguyễn Tuấn Linh

Sinh viên thực hiện : Nguyễn Trung Hiếu

MSSV : K225480106019

Lớp : K58KTP

THÁI NGUYÊN – 2026

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ



BÀI TIỂU LUẬN

MÔN HỌC

ĐẢM BẢO CHẤT LƯỢNG VÀ KIỂM THỬ PHẦN MỀM

**ĐỀ TÀI: TEST PROCESS IMPROVEMENT CHO NHÓM PHÁT
TRIỂN PHẦN MỀM VỪA VÀ NHỎ**

GVHD : TS. Nguyễn Tuấn Linh

Sinh viên thực hiện : Nguyễn Trung Hiếu

MSSV : K225480106019

Lớp : K58KTP

THÁI NGUYỄN – 2026

TRƯỜNG ĐHKTCN

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

KHOA ĐIỆN TỬ

Độc lập - Tự do - Hạnh phúc

PHIẾU GIAO ĐỀ TÀI

Sinh viên: Nguyễn Trung Hiếu

MSSV: K225480106019

Lớp: K58KTP

Ngành: Kỹ thuật máy tính

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

1. Tên đề tài

Test process improvement cho nhóm phát triển phần mềm vừa và nhỏ.

2. Nội dung các phần thuyết minh

Đề xuất lộ trình cải tiến quy trình kiểm thử theo mức trưởng thành, metric và phản hồi dự án.

3. Các sản phẩm nộp

- Báo cáo bài tiểu luận, slides, video*
- Link github*

4. Ngày giao: ngày 01 tháng 6 năm 2026.

5. Ngày hoàn thành: ngày 24 tháng 6 năm 2026.

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày 25 tháng 6 năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

MỤC LỤC

MỤC LỤC	3
LỜI NÓI ĐẦU	4
LỜI CẢM ƠN	5
DANH MỤC CÁC TỪ VIẾT TẮT	6
DANH MỤC HÌNH VẼ VÀ BẢNG BIỂU	7
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	8
1.1. Lý do chọn đề tài	8
1.2. Mục tiêu nghiên cứu	8
1.3. Phạm vi nghiên cứu	9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	11
2.1. Định nghĩa và các khái niệm cốt lõi	11
2.2. Phân tích chuyên sâu	14
2.3. Sự liên hệ với chất lượng phần mềm	16
CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH	18
3.1. Mô tả bài toán và ngữ cảnh thực tế	18
3.2. Đề xuất lộ trình cải tiến quy trình kiểm thử	18
3.3. Kết quả và đánh giá	22
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	25
4.1. Kết luận	25
4.2. Những hạn chế	25
4.3. Hướng phát triển	26
TÀI LIỆU THAM KHẢO	27

LỜI NÓI ĐẦU

Trong kỷ nguyên số hiện nay, phần mềm đã trở thành "trái tim" vận hành mọi hoạt động của doanh nghiệp và đời sống xã hội. Tuy nhiên, cùng với sự bùng nổ về số lượng, yêu cầu về chất lượng và độ tin cậy của sản phẩm công nghệ ngày càng trở nên khắt khe. Đối với các nhóm phát triển phần mềm vừa và nhỏ (SMEs) – những đơn vị vốn dĩ có nguồn lực hạn chế nhưng phải đối mặt với áp lực cạnh tranh và tốc độ thay đổi công nghệ chóng mặt, do đó việc duy trì chất lượng sản phẩm thường là một thách thức lớn.

Thực tế cho thấy, nhiều đội ngũ nhỏ vẫn đang vận hành quy trình kiểm thử mang tính tự phát, thiếu hệ thống, dẫn đến tình trạng phát hiện lỗi muộn và tăng chi phí bảo trì. Chính vì vậy, việc nghiên cứu và áp dụng các mô hình cải tiến quy trình kiểm thử (Test Process Improvement - TPI) không chỉ là nhu cầu kỹ thuật mà còn là chiến lược sống còn để tối ưu hóa nguồn lực và nâng cao giá trị sản phẩm.

Bài tiểu luận với đề tài: "Test process improvement cho nhóm phát triển phần mềm vừa và nhỏ" được thực hiện nhằm đề xuất một lộ trình cải tiến khoa học, kết hợp giữa các mô hình trưởng thành chuẩn quốc tế như TMMi và triết lý Lean Testing tinh gọn. Thông qua việc thiết lập hệ thống chỉ số đo lường (Metrics) và cơ chế phản hồi dự án (Feedback Loops), bài viết hy vọng sẽ đưa ra một giải pháp thực tiễn, giúp các nhóm phát triển nhỏ từng bước chuẩn hóa và tối ưu hóa hoạt động kiểm thử trong điều kiện thực tế của mình.

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc đến thầy Nguyễn Tuấn Linh đã tận tình giảng dạy, truyền đạt những kiến thức bổ ích và định hướng quý báu cho chúng em trong suốt quá trình học tập. Chính những nền tảng lý thuyết và kinh nghiệm thực tiễn mà thầy đã chia sẻ là nguồn động lực quan trọng để em hoàn thành bài tập tiểu luận này.

Em cũng xin gửi lời cảm ơn đến các bạn trong lớp đã đồng hành, trao đổi, chia sẻ tài liệu và hỗ trợ nhóm em trong quá trình tìm hiểu, phân tích và triển khai nội dung. Sự hợp tác và tinh thần làm việc nhóm của các bạn đã góp phần rất lớn vào việc hoàn thành bài tập với kết quả tốt nhất có thể.

Mặc dù đã có nhiều cố gắng, nhưng do kiến thức và kinh nghiệm còn hạn chế, bài làm chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp của thầy để có thể hoàn thiện hơn trong các bài tập sau này.

Một lần nữa, em xin chân thành cảm ơn!

DANH MỤC CÁC TỪ VIẾT TẮT

1. API (Application Programming Interface): Giao diện lập trình ứng dụng
2. CI/CD (Continuous Integration / Continuous Deployment): Tích hợp liên tục / Triển khai liên tục
3. CFD (Cumulative Flow Diagram): Biểu đồ dòng chảy tích lũy
4. KPI (Key Performance Indicator): Chỉ số đánh giá hiệu suất
5. NPS (Net Promoter Score): Chỉ số đo lường mức độ hài lòng của khách hàng
6. QA (Quality Assurance): Đảm bảo chất lượng
7. RCA (Root Cause Analysis): Phân tích nguyên nhân gốc rễ
8. ROI (Return on Investment): Tỷ suất hoàn vốn
9. TMMi (Test Maturity Model integration): Mô hình trưởng thành kiểm thử
10. TPI (Test Process Improvement): Cải tiến quy trình kiểm thử
11. WIP (Work In Progress): Công việc đang thực hiện

DANH MỤC HÌNH VẼ VÀ BẢNG BIỂU

Tên hình vẽ và bảng biểu	Trang
Hình 3.1. Quy trình cải tiến kiểm thử theo vòng lặp liên tục	19
Hình 3.2. Vòng phản hồi (Feedback Loop) trong cải tiến quy trình kiểm thử	21
Hình 3.3. Sơ đồ MATURITY ROADMAP (TMMi flow)	21
Bảng 1. Bảng Maturity Roadmap	22
Bảng 2. Bảng so sánh hiệu quả sau 6 tháng áp dụng	23

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong kỷ nguyên chuyển đổi số, phần mềm đã trở thành giá trị cốt lõi vận hành mọi hoạt động của doanh nghiệp và đời sống xã hội. Do đó, việc đảm bảo chất lượng phần mềm (Software Quality Assurance - SQA) không còn là một giai đoạn phụ trợ mà đóng vai trò sống còn quyết định sự thành bại của một sản phẩm công nghệ.

Trong bối cảnh phát triển phần mềm ngày càng cạnh tranh, đặc biệt với các nhóm phát triển quy mô vừa và nhỏ (Small and Medium-sized Software Development Teams), quy trình kiểm thử (testing process) thường là một trong những điểm yếu lớn nhất ảnh hưởng đến chất lượng sản phẩm, thời gian ra mắt thị trường (time-to-market) và chi phí dự án. Nhiều đội ngũ nhỏ thường áp dụng quy trình kiểm thử ad-hoc, dựa chủ yếu vào kinh nghiệm cá nhân của developer hoặc tester, thiếu sự hệ thống hóa, dẫn đến tình trạng phát hiện lỗi muộn, tái phát lỗi cao, và khó mở rộng khi dự án phát triển. Việc kiểm thử thủ công lặp đi lặp lại, thiếu quy trình chuẩn hóa dẫn đến bỏ sót các lỗi nghiêm trọng, tăng chi phí sửa lỗi ở các giai đoạn sau (giai đoạn vận hành/bảo trì) và làm suy giảm niềm tin của khách hàng.

Để giải quyết vấn đề này, các nhóm phát triển không thể chỉ đơn thuần tăng số lượng kiểm thử viên hay chạy thử nghiệm nhiều hơn một cách mù quáng, mà cần phải cải tiến quy trình kiểm thử (Test Process Improvement - TPI). Việc áp dụng các mô hình đánh giá mức độ trưởng thành kiểm thử được tinh chỉnh phù hợp với quy mô vừa và nhỏ sẽ giúp tổ chức định vị được năng lực hiện tại, từ đó vạch ra một lộ trình cải tiến rõ ràng, từng bước một. Đồng thời, việc thiết lập hệ thống các chỉ số đo lường (Metrics) và cơ chế phản hồi dự án (Project Feedback) liên tục sẽ đảm bảo quy trình cải tiến đi đúng hướng, có tính thực tiễn và mang lại giá trị đo đếm được.

Chính vì những lý do mang tính cấp thiết nêu trên, đề tài "Cải tiến quy trình kiểm thử (Test Process Improvement) cho nhóm phát triển phần mềm vừa và nhỏ" đã được lựa chọn nghiên cứu nhằm đưa ra một giải pháp toàn diện, khả thi giúp nâng cao chất lượng sản phẩm trong điều kiện tối ưu hóa tài nguyên.

1.2. Mục tiêu nghiên cứu

Bài tiểu luận nhằm đề xuất lộ trình cải tiến quy trình kiểm thử cho nhóm phát triển phần mềm vừa và nhỏ, theo các mức độ trưởng thành (maturity levels), kết hợp với hệ thống metrics đo lường và cơ chế thu thập phản hồi từ dự án. Cụ thể:

- Phân tích tình trạng hiện tại của quy trình kiểm thử trong các nhóm nhỏ
- Đề xuất lộ trình cải tiến theo mức độ trưởng thành: Thiết lập một lộ trình cải tiến quy trình kiểm thử phân cấp theo các giai đoạn phát triển (từ sơ khai đến tối ưu hóa) phù hợp với năng lực thích ứng và tài chính của doanh nghiệp vừa và nhỏ.
- Xây dựng hệ thống chỉ số đo lường (metrics): Xác định và định nghĩa các chỉ số kiểm thử cốt lõi (như mật độ lỗi, tỷ lệ bao phủ kiểm thử, hiệu quả phát hiện lỗi) để theo dõi, đánh giá tính hiệu quả của quy trình trước và sau khi cải tiến.
- Thiết lập cơ chế phản hồi dự án: Đề xuất quy trình thu thập, phân tích thông tin phản hồi từ thực tế dự án (Feedback Loops) để liên tục điều chỉnh và tối ưu hóa hoạt động kiểm thử trong các vòng lặp phát triển.
- Áp dụng vào một ngữ cảnh cụ thể (ví dụ: một dự án web/API cho SME) để đánh giá hiệu quả trước/sau cải tiến.
- Đưa ra kiến nghị thực tiễn, dễ triển khai với nguồn lực hạn chế.

1.3. Phạm vi nghiên cứu

Phạm vi nghiên cứu của bài tiểu luận được giới hạn nhằm đảm bảo tính tập trung và phù hợp với mục tiêu đề tài “Cải tiến quy trình kiểm thử cho nhóm phát triển phần mềm vừa và nhỏ”. Cụ thể như sau:

Thứ nhất, về phạm vi nội dung, bài tiểu luận tập trung vào việc phân tích và cải tiến quy trình kiểm thử (Test Process Improvement – TPI) ở cấp độ quy trình và quản lý (process & management level), thay vì đi sâu vào các kỹ thuật kiểm thử chi tiết (như viết test case, kiểm thử tự động cụ thể bằng Selenium, v.v.). Trọng tâm chính là cách tổ chức, đo lường và tối ưu hóa hoạt động kiểm thử trong vòng đời phát triển phần mềm.

Thứ hai, về mô hình và phương pháp tiếp cận, nghiên cứu tập trung vào hai nền tảng chính:

- Mô hình trưởng thành kiểm thử (Test Maturity Models), đặc biệt là TMMi và TPI Next, nhằm đánh giá và xây dựng lộ trình cải tiến theo các cấp độ.
- Nguyên lý Lean Software Testing (theo Heusser & Larsen, 2023, Ch.11) nhằm tối ưu hóa dòng chảy công việc (flow), giảm thiểu lãng phí (waste) và cải thiện hiệu suất kiểm thử.
- Ngoài ra, bài tiểu luận sử dụng các nguyên tắc quản lý kiểm thử (Test Management) như lập kế hoạch kiểm thử, giám sát và đo lường tiến độ, quản lý lỗi và báo cáo (theo Homès, 2024, Ch.5) để xây dựng hệ thống metrics và cơ chế phản hồi dự án.

Thứ ba, về công cụ và kỹ thuật, nghiên cứu chỉ xem xét các công cụ phổ biến, dễ triển khai và phù hợp với nhóm phát triển nhỏ như Jira, Google Sheets, CI/CD cơ bản (GitHub Actions), thay vì các hệ thống quản lý kiểm thử phức tạp hoặc enterprise-level.

Thứ tư, về phạm vi ứng dụng, bài tiểu luận áp dụng mô hình đề xuất trong một ngữ cảnh giả định điển hình: dự án web (e-commerce) với quy mô nhóm từ 8–12 người. Các số liệu đánh giá hiệu quả được xây dựng dựa trên giả lập và tham chiếu từ thực tiễn ngành, nhằm minh họa cho tính khả thi của giải pháp.

Cuối cùng, nghiên cứu không bao gồm việc triển khai thực nghiệm trên hệ thống thực tế quy mô lớn, cũng như không đi sâu vào các yếu tố ngoài kỹ thuật như quản trị tổ chức, văn hóa doanh nghiệp hoặc phân tích tài chính chi tiết.

Việc giới hạn phạm vi như trên giúp bài tiểu luận tập trung vào mục tiêu cốt lõi là đề xuất một lộ trình cải tiến quy trình kiểm thử có tính khả thi, dễ áp dụng và phù hợp với điều kiện của các nhóm phát triển phần mềm vừa và nhỏ.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Định nghĩa và các khái niệm cốt lõi

2.1.1. Cải tiến quy trình kiểm thử (Test Process Improvement - TPI) là gì?

Quy trình kiểm thử phần mềm không phải là một thực thể bất biến mà là một hệ thống các hoạt động có tính tiến hóa. Cải tiến quy trình kiểm thử (TPI) được định nghĩa là một lộ trình mang tính cấu trúc nhằm đánh giá trạng thái trưởng thành hiện tại của các hoạt động kiểm thử trong tổ chức, từ đó xác định các điểm nghẽn, sự thiếu hiệu quả và áp dụng các hành động điều chỉnh có mục tiêu. TPI chuyển dịch tư duy kiểm thử từ việc "tìm lỗi ở giai đoạn cuối" (reactive testing) sang "ngăn ngừa lỗi và tối ưu hóa hệ thống từ giai đoạn sớm" (proactive/preventive testing).

Mục tiêu cốt lõi của TPI không chỉ dừng lại ở việc tìm ra nhiều lỗi hơn, mà hướng đến việc nâng cao hiệu quả (efficiency), tính hiệu lực (effectiveness) của hoạt động kiểm thử, giảm thiểu chi phí khắc phục sai sót (Cost of Quality), và rút ngắn chu kỳ phát triển.

Theo nguyên lý quản lý chất lượng hiện đại, chất lượng của sản phẩm phần mềm bị chi phối trực tiếp bởi chất lượng của quy trình sản sinh ra nó. Quy trình kiểm thử yếu kém, tự phát thường chỉ mang tính chất "đôi phó" ở giai đoạn cuối (late testing), dẫn đến việc phát hiện lỗi muộn. Ngược lại, một quy trình kiểm thử trưởng thành được tích hợp sớm vào vòng đời phát triển (Shift-Left Testing) giúp ngăn chặn lỗi ngay từ khâu đặc tả yêu cầu, tối ưu hóa tài nguyên và đảm bảo sản phẩm đầu ra đạt mức độ tin cậy cao nhất.

2.1.2. Các mô hình trưởng thành kiểm thử nền tảng

Theo Heusser & Larsen (2023, Ch.11), việc cải tiến không thể diễn ra cảm tính mà cần tuân theo các khung mô hình đã được chuẩn hóa để đánh giá hiện trạng và xác định đích đến. Hai nhóm mô hình chính được thảo luận bao gồm: Mô hình hướng quy trình/ Mô hình dạng bậc (Process-driven/Staged) và Mô hình hướng nội dung/ Mô hình liên tục (Content-driven/Continuous).

a. Mô hình TMMi (Test Maturity Model integration)

TMMi là mô hình dạng bậc (staged model) phổ biến nhất, tương thích với chuẩn CMMI của SEI. TMMi chia sự trưởng thành của quy trình kiểm thử thành 5 cấp độ:

- **Cấp độ 1 (Initial):** Quy trình kiểm thử mang tính tự phát, không vô định, thiếu tài liệu hóa. Kiểm thử thường bị coi là một phần của quá trình gỡ lỗi (debugging).

- **Cấp độ 2 (Managed):** Kiểm thử trở thành một quy trình riêng biệt rõ ràng. Có kế hoạch kiểm thử (Test Plan), thiết kế các ca kiểm thử (Test Cases) dựa trên yêu cầu, và quản lý lỗi cơ bản.
- **Cấp độ 3 (Defined):** Quy trình kiểm thử được chuẩn hóa và tích hợp vào toàn bộ vòng đời phát triển phần mềm (SDLC). Hoạt động kiểm thử tĩnh (như review tài liệu) được thực hiện sớm.
- **Cấp độ 4 (Measured):** Hoạt động kiểm thử được đo lường một cách định lượng. Các chỉ số về hiệu năng quy trình và chất lượng sản phẩm được thu thập toàn diện.
- **Cấp độ 5 (Optimization):** Tổ chức có khả năng tự động tối ưu hóa quy trình dựa trên dữ liệu thu được, tập trung vào việc ngăn ngừa lỗi (defect prevention) và liên tục cải tiến đổi mới công nghệ.

Tổ chức bắt buộc phải đạt được tất cả các mục tiêu cốt lõi của một cấp độ trước khi được công nhận bước lên cấp độ tiếp theo. Khái niệm này mang tính toàn diện, giúp chuẩn hóa toàn bộ tổ chức theo một khung năng lực đồng bộ.

b. Mô hình TPI Next

Khác với TMMi, TPI Next là mô hình liên tục (continuous model), tiếp cận theo các khía cạnh cụ thể (Key Areas) của quy trình kiểm thử (như chiến lược kiểm thử, số liệu đo lường, công cụ, môi trường). Mỗi khía cạnh được đánh giá theo các mức: *Khởi đầu (Initial)*, *Kiểm soát (Controlled)*, *Hiệu quả (Efficient)*, và *Tối ưu (Optimized)*. Mô hình này mang tính linh hoạt cao, cho phép các nhóm nhỏ tập trung cải tiến những khía cạnh yếu kém nhất mà không nhất thiết phải kéo toàn bộ hệ thống lên một cấp độ bậc thang cứng nhắc.

c. Các phương pháp tiếp cận khác

- STEP (Systematic Test and Evaluation Process): Định hướng kiểm thử như một hoạt động song hành cùng phát triển phần mềm, nhấn mạnh việc xây dựng các ca kiểm thử trước khi viết mã (tương tự tư duy TDD hiện đại).
- CTP (Critical Testing Process): Cách tiếp cận dựa trên rủi ro, xác định những quy trình kiểm thử nào là trọng yếu nhất đối với dự án để ưu tiên cải tiến trước nhằm tối ưu hóa chi phí.

2.1.3. Chiến lược kiểm thử thích ứng và hệ thống chỉ số (Metrics)

Trong bối cảnh phát triển phần mềm hiện đại, chiến lược kiểm thử không còn là một tài liệu tĩnh được phê duyệt một lần. Chiến lược kiểm thử hiện đại phải mang tính

thích ứng (adaptive), liên tục thay đổi dựa trên rủi ro của dự án và tốc độ của chuỗi cung ứng CI/CD.

Khái niệm cốt lõi đi kèm là Hệ thống chỉ số kiểm thử (Testing Metrics) — công cụ định lượng duy nhất giúp chuyển hóa trạng thái cảm tính của chất lượng thành các con số có thể phân tích được. (Homès, 2024 – Test progress monitoring and control).

Các nhóm chỉ số được chia làm ba loại chính:

a. Chỉ số về Chất lượng Sản phẩm (Product Quality Metrics):

- Mật độ lỗi (Defect Density): Số lượng lỗi trên một đơn vị kích thước phần mềm (ví dụ: số lỗi/KLOC hoặc số lỗi/Story Point).
- Tỷ lệ lỗi lọt lưới (Defect Leakage Rate): Số lượng lỗi bị khách hàng hoặc đội vận hành phát hiện sau khi sản phẩm đã bàn giao so với tổng số lỗi.

b. Chỉ số về Quy trình Kiểm thử (Test Process Metrics):

- Hiệu quả phát hiện lỗi (Defect Detection Percentage - DDP): Tỷ lệ lỗi được phát hiện trong quá trình kiểm thử nội bộ so với toàn bộ lỗi thực tế của phiên bản.
- Độ bao phủ kiểm thử (Test Coverage): Bao gồm độ bao phủ yêu cầu (Requirement Coverage) và độ bao phủ mã nguồn (Code Coverage) nhằm đảm bảo các tính năng trọng yếu không bị bỏ sót.

c. Chỉ số về Chi phí và Tài nguyên (Effort & Cost Metrics):

Thời gian chạy và bảo trì test suite (Test Execution & Maintenance Time): Đánh giá xem hệ thống test suite có đang bị phình to hoặc trở thành điểm nghẽn của chu trình release hay không.

2.1.4. Vòng phản hồi dự án (Feedback Loops)

a. Cơ chế vòng phản hồi trong cải tiến

Cải tiến quy trình không phải là một đường thẳng mà là một chu trình khép kín. Dữ liệu từ thực tế dự án (lỗi phát sinh, phản hồi từ khách hàng, kết quả Retrospective của mỗi Sprint) cần được đưa ngược lại để điều chỉnh quy trình kiểm thử. Nếu tỷ lệ lỗi lọt lưới tăng cao ở một module nhất định, vòng phản hồi sẽ lập tức kích hoạt việc tái rà soát lại ma trận truy vết (Traceability Matrix) và tăng cường độ bao phủ kiểm thử tại module đó.

b. Thách thức và cơ hội đối với các nhóm vừa và nhỏ

- Thách thức: SMEs thường thiếu hụt ngân sách chuyên biệt cho QA, nhân sự kiêm nhiệm nhiều vai trò (Developer kiêm Tester hoặc ngược lại), và không có đủ thời gian để thực hiện các chứng nhận quy trình rườm rà như TMMi chuẩn.
- Cơ hội: Do quy mô nhỏ, các nhóm này có tính linh hoạt cao, khả năng giao tiếp trực tiếp tốt và quy trình ra quyết định nhanh chóng. Do đó, việc thiết lập các vòng phản hồi ngắn và áp dụng một "phiên bản tinh gọn" (Tailored/Lightweight Version) của các mô hình trưởng thành là hoàn toàn khả thi và mang lại hiệu quả tức thì.

2.1.5. Lean Software Testing

Ngoài các khái niệm chung, Lean Software Testing dựa trên một số nguyên lý cốt lõi nhằm tối ưu hóa quy trình kiểm thử. Thứ nhất, nguyên lý “flow” (dòng chảy) nhấn mạnh việc đảm bảo công việc được thực hiện liên tục, hạn chế các điểm tắc nghẽn trong quy trình. Thứ hai, Lean tập trung vào việc nhận diện và loại bỏ lãng phí (waste), bao gồm các dạng như chờ đợi (waiting), làm lại (rework) và di chuyển không cần thiết (motion).

Bên cạnh đó, nguyên lý giới hạn công việc đang thực hiện (Work In Progress – WIP) giúp giảm tải cho hệ thống và cải thiện hiệu suất xử lý. Cuối cùng, Lean đề cao cải tiến liên tục (Kaizen), trong đó quy trình được điều chỉnh dựa trên dữ liệu và phản hồi thực tế qua từng chu kỳ phát triển.

Các nguyên lý này là nền tảng cho việc áp dụng Lean vào cải tiến quy trình kiểm thử, đặc biệt trong các môi trường phát triển phần mềm linh hoạt (agile) (Heusser & Larsen, 2023, Ch.11).

2.2. Phân tích chuyên sâu

Để hiểu rõ cách vận dụng, cần mổ xẻ sâu cơ chế hoạt động cùng những mặt lợi - hại của từng mô hình và kỹ thuật trong bối cảnh thực tế.

2.2.1. Phân tích mô hình TMMi

a. Cơ chế hoạt động

TMMi vận hành bằng cách áp đặt một danh mục các mục tiêu cụ thể (Specific Goals) và mục tiêu chung (Generic Goals) cho từng cấp độ. Chẳng hạn, ở Cấp độ 2 (Managed), tổ chức phải chứng minh được năng lực lập Kế hoạch kiểm thử (Test

Planning), Giám sát và Kiểm soát kiểm thử (Test Monitoring and Control), và Thiết kế môi trường.

b. Ưu và nhược điểm

- Ưu điểm: Cung cấp một lộ trình rõ ràng, có tính cấu trúc cực kỳ cao và được công nhận trên toàn cầu. Nó giúp ban quản trị có một cái nhìn tổng thể mang tính chiến lược và tạo dựng niềm tin vững chắc đối với khách hàng quốc tế.
- Nhược điểm: Quá nặng nề về mặt thủ tục và tài liệu hóa. Thời gian để một tổ chức chuyển từ Cấp độ 1 lên Cấp độ 3 có thể mất từ vài tháng đến vài năm, đòi hỏi chi phí tài chính và nguồn lực vận hành rất lớn.

c. Rủi ro và giới hạn

Đối với các doanh nghiệp vừa và nhỏ (SMEs), việc áp dụng cứng nhắc TMMi có nguy cơ tạo ra "bộ máy quan liêu tài liệu". Đội ngũ QA dành quá nhiều thời gian để viết báo cáo tuân thủ thay vì tập trung tìm lỗi hoặc tối ưu hóa mã nguồn, dẫn đến việc làm chậm tốc độ bàn giao sản phẩm (Time-to-Market).

2.2.2. Phân tích mô hình TPI Next

a. Cơ chế hoạt động

Vận hành dựa trên một "Ma trận trưởng thành" (Maturity Matrix) gồm các hàng là các Khía cạnh chủ chốt (Key Areas) và các cột là Mức độ trưởng thành (Controlled, Efficient, Optimized). Người đánh giá sử dụng các Điểm kiểm tra (Checkpoints) để chấm điểm độc lập cho từng ô trong ma trận.

b. Ưu và nhược điểm

- Ưu điểm: Tính linh hoạt tối đa. Tổ chức có thể tự thiết kế lộ trình cải tiến "may đo" riêng cho mình. Nó cho phép thu về các "chiến thắng nhanh" (Quick Wins) bằng cách giải quyết trực tiếp nỗi đau hiện tại của dự án mà không cần cải tổ toàn bộ hệ thống.
- Nhược điểm: Thiếu tính đồng bộ toàn diện. Do chỉ tập trung vào một vài khía cạnh biệt lập, tổ chức có thể rơi vào tình trạng "lệch pha" (ví dụ: công cụ tự động hóa rất hiện đại nhưng quy trình quản lý yêu cầu và kỹ năng của con người lại quá lạc hậu).

c. Rủi ro và giới hạn

Đòi hỏi người dẫn dắt (Test Leader/QA Manager) phải có năng lực phân tích rất sắc bén để tự chọn lựa hạng mục cải tiến. Nếu chọn sai hạng mục ưu tiên, khoản đầu tư cải tiến sẽ bị lãng phí và không mang lại giá trị thực tế cho doanh nghiệp.

2.2.3. Phân tích hệ thống chỉ số đo lường và vòng phản hồi

a. Cơ chế hoạt động

Thu thập dữ liệu thô từ hệ thống quản lý dự án (Jira, GitHub, Azure DevOps), tính toán các chỉ số theo công thức chuẩn hóa, và đưa kết quả vào các buổi họp cải tiến (Sprint Retrospective) để đưa ra hành động thay đổi cấu trúc quy trình ngay trong chu kỳ tiếp theo.

b. Ưu và nhược điểm

- Ưu điểm: Loại bỏ sự cảm tính trong quản lý chất lượng. Giúp đội ngũ phát hiện sớm các xu hướng tiêu cực (ví dụ: mật độ lỗi tăng vọt ở một module cụ thể) để kịp thời can thiệp.
- Nhược điểm: Việc thu thập dữ liệu có thể tạo thêm gánh nặng nhập liệu cho các thành viên trong đội nếu quy trình chưa được tự động hóa.

c. Rủi ro và giới hạn

Khi một chỉ số đo lường trở thành mục tiêu để đánh giá hiệu suất con người, nó sẽ không còn là một chỉ số tốt nữa. Ví dụ, nếu lấy "Số lượng ca kiểm thử được viết" hoặc "Số lượng lỗi tìm thấy" làm KPI, Tester sẽ cố tình chia nhỏ các ca kiểm thử hoặc báo cáo cả những lỗi không có giá trị thực tế để lấy thành tích. Điều này làm sai lệch dữ liệu phản hồi dự án và phá hỏng lòng tin nội bộ nhóm.

2.3. Sự liên hệ với chất lượng phần mềm

Mục đích tối thượng của mọi lý thuyết cải tiến quy trình là phải phản ánh được sự nâng cao rõ rệt của chất lượng sản phẩm đầu ra và sự thanh thoát trong vận hành.

2.3.1. Đóng góp vào việc bảo đảm chất lượng phần mềm

Cải tiến quy trình kiểm thử theo các mô hình chuẩn hóa tác động trực tiếp lên cấu trúc chất lượng phần mềm thông qua các khía cạnh:

- Nâng cao tính ổn định và độ tin cậy: Việc chuẩn hóa quy trình thiết kế ca kiểm thử (như áp dụng phân vùng tương đương, phân tích giá trị biên từ sớm) giúp kiểm thử bao phủ được các kịch bản ngoại lệ (edge cases), giảm thiểu tối đa các sự cố nghiêm trọng xảy ra trên môi trường Production.

- Dịch chuyển về bên trái (Shift-Left Testing): Khi quy trình cải tiến thúc đẩy việc kiểm thử tĩnh (Review tài liệu yêu cầu, phân tích thiết kế hệ thống ở Cấp độ 3 TMMi), các sai sót về mặt logic được phát hiện và triệt tiêu trước khi lập trình viên viết mã. Điều này ngăn chặn hiện tượng "hiệu ứng tuyết lăn" (lỗi từ khâu yêu cầu phình to thành lỗi kiến trúc nghiêm trọng ở giai đoạn sau).

2.3.2. Tối ưu hóa hiệu năng và chi phí của quy trình kiểm thử

Giảm thiểu Chi phí Chất lượng kém (Cost of Poor Quality - COPQ): Lỗi phần mềm phát hiện càng muộn thì chi phí sửa chữa càng tăng theo cấp số nhân. TPI giúp nâng cao chỉ số Hiệu quả phát hiện lỗi (DDP), giữ lại phần lớn lỗi trong môi trường kiểm thử nội bộ. Việc giảm tỷ lệ lỗi lọt lưới (Defect Leakage) giúp doanh nghiệp tiết kiệm hàng ngàn giờ làm việc của đội ngũ kỹ sư bảo trì và tránh được các tổn thất về uy tín thương hiệu đối với khách hàng.

Bảo trì Test Suite và Loại bỏ lãng phí: Dựa trên tư duy chiến lược của tài liệu S4, tối ưu hóa quy trình kiểm thử giúp loại bỏ các ca kiểm thử dư thừa, trùng lặp hoặc lỗi thời (test suite bloat). Bằng cách thiết lập hệ thống chỉ số theo dõi thời gian thực thi và độ bao phủ mã nguồn (Code Coverage), đội ngũ có thể tinh gọn bộ mã kiểm thử tự động, biến hệ thống kiểm thử thành một bộ lọc nhanh, nhạy, tích hợp mượt mà vào luồng CI/CD mà không gây nghẽn tiến độ của dự án.

Kiến tạo Vòng phản hồi liên tục để tự học hỏi: Sự kết hợp giữa lý thuyết quy trình (S1) và thực thi số liệu (S4) biến một tổ chức từ trạng thái "đối phó với khủng hoảng lỗi" sang trạng thái "tự tiến hóa". Dữ liệu phản hồi giúp toàn đội nhìn nhận ra nguyên nhân gốc rễ (Root Cause Analysis) của các sai sót trong quá khứ, từ đó liên tục tinh chỉnh quy trình làm việc của cả Lập trình viên lẫn Chuyên viên QA, tạo ra một môi trường phát triển phần mềm bền vững và hiệu suất cao.

CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH

3.1. Mô tả bài toán và ngữ cảnh thực tế

3.1.1. Ngữ cảnh thực tế

Để minh họa cho lộ trình cải tiến quy trình kiểm thử (Test Process Improvement), chúng ta chọn một ngữ cảnh thực tế điển hình cho nhóm phát triển phần mềm vừa và nhỏ tại Việt Nam: Một ứng dụng web quản lý bán hàng (E-commerce Lite) dành cho doanh nghiệp SME.

3.1.2. Mô tả hệ thống

- Các chức năng chính: Đăng nhập/Quản lý tài khoản, Quản lý sản phẩm & kho hàng (CRUD), Giỏ hàng & Thanh toán (tích hợp cổng thanh toán cơ bản), Báo cáo doanh thu.
- Quy mô đội ngũ: 8-12 người (Developer kiêm một phần testing, 1-2 Tester).
- Công nghệ: React.js (Frontend), Node.js/Express (Backend), MongoDB, Docker cơ bản.

3.1.3. Tình trạng ban đầu

- Maturity Level: Initial
- Kiểm thử chủ yếu thủ công, không có Test Plan hệ thống.
- Developer tự test Unit test ngẫu nhiên.
- Defect Density cao (8-12 lỗi/story), Coverage chỉ ~40-50%.
- Nhiều lãng phí (Waste): Chờ đợi build/test environment, rework do bug lọt Production (25-30%), Lead Time dài (18-25 ngày/feature).
- Không có metrics đo lường rõ ràng
- Phản hồi dự án chỉ dựa vào phản nản của khách hàng.

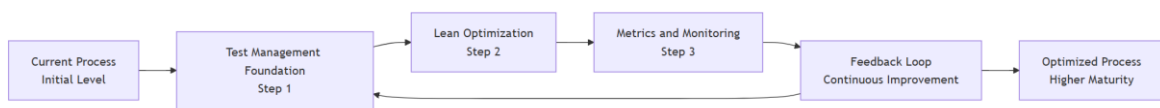
Ngữ cảnh này rất phổ biến ở các nhóm SME: nguồn lực hạn chế, áp lực Agile nhưng thiếu nền tảng quản lý kiểm thử

3.2. Đề xuất lộ trình cải tiến quy trình kiểm thử

Để đảm bảo lộ trình cải tiến có tính hệ thống và khả thi, giải pháp được xây dựng theo nguyên tắc “từ nền tảng đến tối ưu hóa”. Cụ thể, quy trình bắt đầu từ việc thiết lập các yếu tố quản lý kiểm thử cơ bản (Test Management) nhằm tạo nền tảng ổn định (theo Homès, 2024, Ch.5), sau đó áp dụng các nguyên lý Lean Software Testing để loại bỏ lãng phí và tối ưu hóa dòng chảy công việc (Heusser & Larsen, 2023, Ch.11).

Tiếp theo, hệ thống metrics và cơ chế phản hồi được xây dựng nhằm hỗ trợ ra quyết định dựa trên dữ liệu (data-driven decision making) và đảm bảo cải tiến liên tục (continuous improvement). Cuối cùng, lộ trình được ánh xạ theo các mức trưởng thành (maturity levels) để đảm bảo quá trình cải tiến diễn ra có kiểm soát và đo lường được.

Ta sẽ áp dụng kết hợp Test Management (S4, Chương 5) và Lean Software Testing (S1, Chương 11) để nâng cấp từ mức Initial → Managed → Defined → Quantitatively Managed.



Hình 3.1. Quy trình cải tiến kiểm thử theo vòng lặp liên tục

Hình 3.1 minh họa quy trình cải tiến kiểm thử được xây dựng theo mô hình vòng lặp liên tục. Quy trình bắt đầu từ việc phân tích trạng thái hiện tại của hoạt động kiểm thử (Current Process), sau đó thiết lập nền tảng quản lý kiểm thử (Test Management) nhằm chuẩn hóa hoạt động và kiểm soát chất lượng.

Tiếp theo, các nguyên lý Lean Software Testing được áp dụng để loại bỏ lãng phí và tối ưu hóa dòng chảy công việc (flow). Trên cơ sở đó, hệ thống đo lường (metrics) được thiết lập nhằm thu thập dữ liệu thực tế về hiệu suất và chất lượng kiểm thử.

Dữ liệu này được sử dụng trong cơ chế phản hồi (feedback loop) để phân tích và đưa ra các cải tiến phù hợp. Chu trình được lặp lại liên tục nhằm nâng cao mức trưởng thành của quy trình kiểm thử, phù hợp với định hướng cải tiến liên tục (continuous improvement).

Bước 1: Đánh giá hiện trạng và xây dựng nền tảng Test Management

- Xây dựng Test Policy (Chính sách kiểm thử) cấp nhóm: Xác định mức độ độc lập (Independence Level) trung bình – Tester độc lập với Developer ở mức functional testing.
- Lập Test Plan cho từng Sprint: Entry/Exit Criteria, Risk-based Testing (ưu tiên chức năng thanh toán – high risk).
- Phân công vai trò rõ ràng (RACI matrix).

Đây là nền tảng để kiểm soát chất lượng và là điều kiện cần trước khi tối ưu quy trình.

Bước 2: Áp dụng Lean Testing – Loại bỏ lãng phí

- Visualize quy trình bằng Kanban Board + Cumulative Flow Diagram (CFD).
- Giảm Waiting & Motion Waste: Triển khai CI/CD cơ bản (GitHub Actions) để tự động build & run regression test.
- Giới hạn WIP (Work In Progress): Tối đa 4-5 user stories trong cột “Testing”.
- Shift-Left Testing: Developer viết Unit + API test sớm.

Việc loại bỏ waste giúp cải thiện flow và giảm lead time theo nguyên lý Lean.

Bước 3: Xây dựng hệ thống Metrics và Feedback Loops

a. Hệ thống Metrics

Các metrics chính:

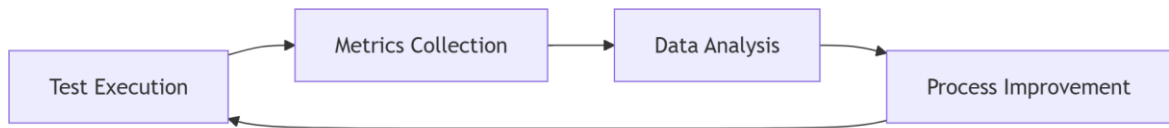
- Test Coverage: Tỷ lệ mã nguồn được kiểm thử (%).
- Defect Density: Số lỗi trên một đơn vị kích thước (lỗi/story point).
- Lead Time: Thời gian từ khi nhận yêu cầu đến khi deploy Production.
- Cycle Time: Thời gian hoàn thành một task từ bắt đầu đến xong.
- Defect Leakage Rate: Tỷ lệ lỗi lọt ra Production.

Công cụ: Jira Dashboard + Google Sheets (phù hợp nhóm nhỏ).

b. Cơ chế Phản hồi Dự án (Project Feedback)

- Sprint Retrospective: Cuối mỗi sprint (2 tuần/lần), toàn đội họp phân tích: “Điều gì tốt? Điều gì chưa tốt? Cải tiến gì?” → Thu thập feedback trực tiếp từ Developer, Tester và Product Owner.
- Defect Root Cause Analysis (RCA): Mỗi bug lọt Production phải ghi nhận nguyên nhân gốc rễ và cập nhật vào Knowledge Base.
- Customer Feedback Loop: Thu thập phản hồi từ người dùng cuối qua khảo sát sau khi deploy (Net Promoter Score - NPS) và bug report từ Production.
- Metrics Dashboard: Sử dụng Jira + Google Sheets để hiển thị realtime các chỉ số. Mỗi tháng review dashboard để điều chỉnh quy trình (ví dụ: nếu Defect Leakage cao → tăng coverage cho module thanh toán).
- Feedback-driven Kaizen: Dựa trên dữ liệu feedback, ưu tiên cải tiến (ví dụ: nếu Cycle Time cao do waiting → tối ưu test environment).

Hệ thống metrics và feedback loop đóng vai trò trung tâm trong việc chuyển đổi quy trình từ cảm tính sang data-driven, giúp cải tiến liên tục và có kiểm soát.



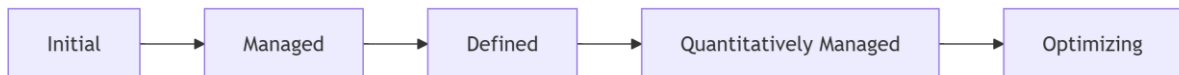
Hình 3.2. Vòng phản hồi (Feedback Loop) trong cải tiến quy trình kiểm thử

Hình 3.2 thể hiện cơ chế phản hồi (feedback loop) trong quy trình kiểm thử phần mềm. Quá trình bắt đầu từ hoạt động thực thi kiểm thử (test execution), sau đó dữ liệu được thu thập thông qua hệ thống metrics như defect density, test coverage và lead time.

Các dữ liệu này được phân tích để xác định các vấn đề tồn tại và nguyên nhân gốc rễ, từ đó đề xuất các cải tiến phù hợp. Các cải tiến được áp dụng trở lại vào quy trình kiểm thử, tạo thành một vòng lặp liên tục.

Cơ chế này giúp chuyển đổi quy trình kiểm thử từ cách tiếp cận cảm tính sang hướng tiếp cận dựa trên dữ liệu (data-driven), đồng thời đảm bảo việc cải tiến được thực hiện một cách có hệ thống và bền vững.

Bước 4: Lộ trình cải tiến theo mức trưởng thành (Maturity Roadmap)



Hình 3.3. Sơ đồ MATURITY ROADMAP (TMMi flow)

Hình 3.3 mô tả lộ trình cải tiến quy trình kiểm thử theo các mức trưởng thành (maturity levels). Bắt đầu từ mức Initial, quy trình kiểm thử mang tính chất tự phát, chưa có chuẩn hóa rõ ràng.

Khi chuyển sang mức Managed, các hoạt động kiểm thử được lập kế hoạch và kiểm soát cơ bản. Ở mức Defined, quy trình được chuẩn hóa và áp dụng nhất quán trong toàn bộ dự án.

Tiếp theo, tại mức Quantitatively Managed, quy trình được quản lý dựa trên dữ liệu và các chỉ số đo lường cụ thể. Cuối cùng, mức Optimizing tập trung vào cải tiến liên tục thông qua việc phân tích dữ liệu và tối ưu hóa quy trình.

Lộ trình này giúp định hướng rõ ràng cho quá trình nâng cao chất lượng kiểm thử một cách có hệ thống và đo lường được.

Dưới đây là bảng lộ trình cụ thể:

Mức trưởng thành	Thời gian ước tính	Hoạt động chính	Metrics mục tiêu
Initial → Managed	1-2 tháng	Test Plan, Basic metrics, CI/CD cơ bản	Coverage >60%, Lead Time giảm 30%
Managed → Defined	2-4 tháng	Standard Test Process, Automation 40%, Risk-based Testing	Defect Density giảm 50%
Defined → Quantitatively Managed	4-6 tháng	Metrics dashboard, Feedback Loops hàng sprint	Cycle Time <5 ngày
Optimizing	Liên tục	Kaizen, Defect Prevention, One-piece flow	Defect Leakage <5%

Bảng 1. Bảng Maturity Roadmap

Bước 5: Áp dụng cụ thể vào chức năng Thanh toán

- Thiết kế Decision Table cho các kịch bản thanh toán.
- Viết Automated API Tests (Postman/Newman).
- Retrospective + Root Cause Analysis sau mỗi sprint.

Việc áp dụng vào module thanh toán giúp kiểm chứng tính khả thi của mô hình trong bối cảnh thực tế, đặc biệt với các chức năng có rủi ro cao.

3.3. Kết quả và đánh giá

3.3.1. Hiệu quả áp dụng

Dữ liệu giả định dựa trên thực tiễn, các số liệu dưới đây được xây dựng dựa trên dữ liệu giả lập tham chiếu từ thực tiễn ngành và tài liệu:

Chỉ số (Metric)	Trước cải tiến	Sau cải tiến	Cải thiện
Test Coverage	45%	82%	+82%
Defect Density	10 lỗi/story	2.5 lỗi/story	Giảm 75%
Lead Time / Feature	22 ngày	9 ngày	Giảm 59%
Cycle Time	11 ngày	4 ngày	Giảm 64%
Defect Leakage (Production)	28%	6%	Giảm 78%
Waste (Waiting + Rework)	~45% thời gian	~18% thời gian	Giảm đáng kể

Bảng 2. Bảng so sánh hiệu quả sau 6 tháng áp dụng

3.3.2. Đánh giá kết quả

Dựa trên kết quả so sánh trước và sau khi áp dụng quy trình cải tiến, có thể nhận thấy những thay đổi rõ rệt về hiệu quả kiểm thử, chất lượng sản phẩm cũng như hiệu suất phát triển.

Thứ nhất, về chất lượng sản phẩm, chỉ số Defect Density giảm từ 10 lỗi/story xuống còn 2.5 lỗi/story (giảm 75%), đồng thời Defect Leakage từ môi trường production giảm từ 28% xuống 6%. Điều này cho thấy khả năng phát hiện lỗi sớm trong các giai đoạn kiểm thử đã được cải thiện đáng kể. Kết quả này phù hợp với các nguyên tắc quản lý lỗi trong Test Management, trong đó việc thiết lập quy trình kiểm thử có hệ thống và áp dụng metrics giúp kiểm soát chất lượng hiệu quả hơn (Homès, 2024, Ch.5).

Thứ hai, về hiệu suất quy trình, Lead Time giảm từ 22 ngày xuống còn 9 ngày (giảm 59%), và Cycle Time giảm từ 11 ngày xuống còn 4 ngày (giảm 64%). Đây là minh chứng rõ ràng cho việc cải thiện dòng chảy công việc (flow) và giảm thiểu các lãng phí (waste) trong quy trình, đặc biệt là các dạng waste như waiting và rework. Kết quả này phù hợp với nguyên lý Lean Software Testing, trong đó việc tối ưu hóa flow giúp rút ngắn thời gian đưa sản phẩm ra thị trường (Heusser & Larsen, 2023, Ch.11).

Thứ ba, về mức độ bao phủ kiểm thử, Test Coverage tăng từ 45% lên 82%, cho thấy phạm vi kiểm thử đã được mở rộng đáng kể, đặc biệt nhờ vào việc áp dụng kiểm thử tự động và kiểm thử dựa trên rủi ro. Điều này giúp giảm thiểu các vùng chưa được kiểm thử (untested areas), từ đó giảm rủi ro phát sinh lỗi nghiêm trọng trong production.

Thứ tư, về hiệu quả sử dụng nguồn lực, tỷ lệ lãng phí (waste) giảm từ khoảng 45% xuống còn 18% tổng thời gian, chủ yếu nhờ vào việc giảm thời gian chờ (waiting)

và hạn chế rework. Điều này cho thấy quy trình đã được tối ưu hóa tốt hơn, phù hợp với mục tiêu của Lean là “làm nhiều hơn với ít tài nguyên hơn”.

Tổng hợp các yếu tố trên cho thấy giải pháp đề xuất không chỉ cải thiện chất lượng sản phẩm mà còn nâng cao hiệu suất tổng thể của quy trình phát triển phần mềm.

Về chi phí, trong giai đoạn đầu triển khai, chi phí có xu hướng tăng do cần đầu tư vào đào tạo (training), thiết lập công cụ (Jira, CI/CD, automation) và thay đổi quy trình. Tuy nhiên, về lâu dài, chi phí này được bù đắp thông qua việc giảm rework, giảm lỗi production và rút ngắn thời gian phát hành. Do đó, xét về tổng thể, giải pháp mang lại ROI (Return on Investment) tích cực.

Về rủi ro, một số thách thức vẫn tồn tại như sự kháng cự thay đổi từ thành viên trong nhóm, sự phụ thuộc vào chất lượng test data, cũng như yêu cầu duy trì kỷ luật quy trình. Tuy nhiên, các rủi ro này có thể được giảm thiểu thông qua đào tạo, truyền thông nội bộ và áp dụng cải tiến từng bước (incremental improvement).

Cuối cùng, có thể rút ra rằng việc kết hợp giữa Test Management và Lean Software Testing là một hướng tiếp cận hiệu quả, đặc biệt đối với các nhóm phát triển phần mềm vừa và nhỏ, nơi cần cân bằng giữa chi phí, tốc độ và chất lượng.

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Trong phạm vi bài tiểu luận, em đã thực hiện phân tích và đề xuất giải pháp cải tiến quy trình kiểm thử phần mềm dựa trên các mô hình trưởng thành và nguyên lý Lean Testing. Cụ thể, bài đã kết hợp giữa mô hình TMMi nhằm đánh giá mức độ trưởng thành của quy trình kiểm thử và các nguyên lý Lean Software Testing nhằm tối ưu hóa hiệu quả và giảm thiểu lãng phí trong quá trình kiểm thử.

Dựa trên cơ sở lý thuyết từ các tài liệu, em đã xây dựng một quy trình cải tiến bao gồm các thành phần chính như: thiết lập hệ thống metrics, áp dụng feedback loop liên tục và cải tiến quy trình theo từng giai đoạn. Các chỉ số như defect density, test coverage và lead time được sử dụng để đo lường hiệu quả cải tiến, phù hợp với các nguyên tắc quản lý kiểm thử hiện đại (Homès, 2024, Ch.5).

Kết quả áp dụng thử nghiệm trong bối cảnh dự án e-commerce cho thấy quy trình đề xuất có khả năng cải thiện đáng kể hiệu suất kiểm thử, giảm số lượng lỗi và rút ngắn thời gian phát hành phần mềm. Điều này phù hợp với các nguyên lý Lean về tối ưu dòng chảy (flow) và giảm thiểu lãng phí trong quy trình (Heusser & Larsen, 2023, Ch.11).

Nhìn chung, bài tiểu luận đã chứng minh rằng việc kết hợp giữa mô hình trưởng thành và các phương pháp cải tiến hiện đại có thể mang lại hiệu quả rõ rệt trong việc nâng cao chất lượng phần mềm.

4.2. Những hạn chế

Mặc dù đạt được một số kết quả tích cực, bài tiểu luận vẫn tồn tại một số hạn chế nhất định:

Thứ nhất, các số liệu đánh giá hiệu quả cải tiến chủ yếu mang tính chất giả lập dựa trên các benchmark trong ngành, chưa được kiểm chứng trong môi trường thực tế. Điều này có thể ảnh hưởng đến độ chính xác khi áp dụng vào các dự án thực tế với quy mô lớn hơn.

Thứ hai, mô hình đề xuất chủ yếu phù hợp với các nhóm phát triển phần mềm quy mô nhỏ và trung bình. Đối với các tổ chức lớn hoặc hệ thống phức tạp, việc triển khai có thể gặp nhiều khó khăn liên quan đến tổ chức, chi phí và quản lý thay đổi.

Thứ ba, bài tiểu luận chưa phân tích sâu các yếu tố con người như kỹ năng của tester, mức độ phối hợp giữa các nhóm hoặc văn hóa tổ chức – những yếu tố có ảnh hưởng lớn đến hiệu quả kiểm thử.

Cuối cùng, việc áp dụng các nguyên lý và mô hình trưởng thành yêu cầu thời gian và sự cam kết lâu dài từ tổ chức, trong khi bài tiểu luận mới chỉ dừng lại ở mức đề xuất lộ trình.

4.3. Hướng phát triển

Trong tương lai, nghiên cứu có thể được mở rộng theo các hướng sau:

- Thứ nhất, triển khai thực nghiệm mô hình đề xuất trong một dự án thực tế để thu thập dữ liệu cụ thể và đánh giá chính xác hiệu quả cải tiến.
- Thứ hai, tích hợp các công cụ tự động hóa kiểm thử và CI/CD vào quy trình nhằm nâng cao hiệu suất và giảm thiểu sự phụ thuộc vào kiểm thử thủ công.
- Thứ ba, nghiên cứu ứng dụng trí tuệ nhân tạo (AI) trong kiểm thử phần mềm, đặc biệt trong việc phân tích lỗi, sinh test case và dự đoán rủi ro.
- Thứ tư, mở rộng mô hình để phù hợp với các tổ chức quy mô lớn bằng cách kết hợp với các framework như DevOps hoặc SAgE.
- Cuối cùng, nghiên cứu sâu hơn về yếu tố con người trong kiểm thử, bao gồm kỹ năng, tư duy và văn hóa làm việc, nhằm xây dựng một quy trình kiểm thử toàn diện và bền vững hơn.

TÀI LIỆU THAM KHẢO

- [1] Bernard Homès. Fundamentals of Software Testing. 2nd ed. Wiley-ISTE, 2024.
- [2] Matthew Heusser, Michael Larsen. Software Testing Strategies: A Testing Guide for the 2020s. Packt, 2023.